

EARLY — Full VPS Setup Guide

Single IONOS VPS serving frontend + backend + database + retraining pipeline. DNS managed by GoDaddy. Domain: early.gregtutors.com.

DNS Authority

■ gregtutors.com nameservers are ns75.domaincontrol.com and ns76.domaincontrol.com — GoDaddy controls DNS. All DNS changes are made in GoDaddy only. IONOS is the VPS host only — no DNS changes needed there.

Architecture Overview

```
early.gregtutors.com (one domain, one server, one SSL cert) | Nginx |-- / -->
/app/early/dist (static frontend, HTML/JS) |-- /api/ --> FastAPI :8000 (samples, model,
sessions) |-- /model/ --> /app/model/ (weights JSON, static file) | FastAPI (Python,
systemd-managed) |-- POST /api/samples receive MFCC batch at session end |-- GET
/api/model serve latest model weights |-- POST /api/sessions log per-phoneme session
outcomes | PostgreSQL |-- mfcc_samples raw training vectors |-- sessions per-phoneme
accuracy logs |-- model_versions retraining history | Cron (2am nightly) |-- retrain.py
loads vectors, retrains, saves weights JSON
```

No CORS issues — frontend and backend share the same domain. No Vercel dependency. Fixed cost ~\$6/month regardless of usage.

Step 1 — DNS Setup (GoDaddy)

Do this before reinstalling the VPS so DNS has time to propagate while you work through the remaining steps.

■ Get your IONOS VPS IP address from the IONOS control panel before starting. The IP stays the same after a reinstall.

In GoDaddy:

- Log in to GoDaddy and go to My Products
- Find gregtutors.com and click DNS
- Click Add New Record
- Fill in the fields as shown below
- Click Save

Field	Value
Type	A
Name	early

Value	<your IONOS VPS IP address>
TTL	1 hour (3600)

Result: early.gregtutors.com will point to your IONOS VPS. GoDaddy DNS typically propagates within a few minutes to an hour.

Confirm propagation before running certbot in Step 9:

```
# run this from PowerShell or any terminal nslookup early.gregtutors.com # should return
your IONOS VPS IP # if it returns nothing or the wrong IP, wait and try again
```

Step 2 — Reinstall VPS Image (IONOS)

In the IONOS control panel, reinstall the VPS with Ubuntu 22.04 LTS. Do not use Ubuntu 24 — 22.04 LTS has the widest package support and security patches until 2027.

■ Reinstalling wipes everything. The gregtutors.com tutoring site will go offline. Confirm you do not need anything from it first.

■ IONOS will email you new root credentials after the reinstall completes. Keep these handy for the next step.

Step 3 — First Login & Security

SSH in as root using the credentials IONOS emailed. Do all of the following before anything else.

3a. Update the system

```
ssh root@ apt update && apt upgrade -y
```

3b. Create a non-root user

```
adduser early usermod -aG sudo early
```

3c. Copy SSH key to new user, disable root login

```
# copy your SSH key to the new user rsync --archive --chown=early:early ~/.ssh /home/early
# disable root SSH and password login nano /etc/ssh/sshd_config # set: PermitRootLogin no
# set: PasswordAuthentication no systemctl restart sshd
```

■ Open a new terminal and confirm SSH works as the early user before closing your root session. If you lock yourself out you will need to use the IONOS console to recover.

```
ssh early@ # test this in a new terminal first
```

Step 4 — Install All Dependencies

```
sudo apt install -y \ nginx \ postgresql postgresql-contrib \ python3 python3-pip
python3-venv \ git \ certbot python3-certbot-nginx \ ufw
```

Firewall

```
sudo ufw allow OpenSSH sudo ufw allow 'Nginx Full' sudo ufw enable sudo ufw status
```

Step 5 — PostgreSQL Setup

5a. Create database and user

```
sudo -u postgres psql CREATE DATABASE earlydb; CREATE USER earlyuser WITH PASSWORD 'choose-a-strong-password'; GRANT ALL PRIVILEGES ON DATABASE earlydb TO earlyuser; \q
```

5b. Create tables

```
psql -U earlyuser -d earlydb -h localhost CREATE TABLE mfcc_samples ( id SERIAL PRIMARY KEY, student_id TEXT NOT NULL, word TEXT NOT NULL, phoneme TEXT NOT NULL, class_idx INT NOT NULL, correct BOOLEAN NOT NULL, mfcc FLOAT8[] NOT NULL, confidence FLOAT4, recorded_at TIMESTAMPTZ DEFAULT NOW() ); CREATE TABLE sessions ( id SERIAL PRIMARY KEY, student_id TEXT NOT NULL, unit INT NOT NULL, stage INT NOT NULL, phoneme TEXT NOT NULL, attempts INT NOT NULL, correct INT NOT NULL, accuracy FLOAT4 NOT NULL, session_date DATE DEFAULT CURRENT_DATE ); CREATE TABLE model_versions ( id SERIAL PRIMARY KEY, trained_at TIMESTAMPTZ DEFAULT NOW(), sample_count INT, val_accuracy FLOAT4, weights_path TEXT, active BOOLEAN DEFAULT FALSE );
```

Step 6 — App Directory Structure

```
sudo mkdir -p /app/early/dist # frontend static files sudo mkdir -p /app/api # FastAPI backend sudo mkdir -p /app/model # model weights JSON sudo mkdir -p /app/logs # application logs sudo chown -R early:early /app # clone your GitHub repo cd /app/early git clone https://github.com/yourusername/early.git . # python virtual environment cd /app/api python3 -m venv venv source venv/bin/activate pip install fastapi uvicorn psycpg2-binary numpy scikit-learn
```

Step 7 — FastAPI Backend

Create /app/api/main.py:

```
from fastapi import FastAPI from pydantic import BaseModel from typing import List import psycpg2, json, os app = FastAPI() DB = os.environ['DATABASE_URL'] class Sample(BaseModel): student_id: str word: str phoneme: str class_idx: int correct: bool mfcc: List[float] confidence: float class SessionLog(BaseModel): student_id: str unit: int stage: int phoneme: str attempts: int correct: int accuracy: float @app.post('/api/samples') def store_samples(samples: List[Sample]): conn = psycpg2.connect(DB) cur = conn.cursor() for s in samples: cur.execute( 'INSERT INTO mfcc_samples ' '(student_id,word,phoneme,class_idx,correct,mfcc,confidence) ' 'VALUES (%s,%s,%s,%s,%s,%s,%s)', (s.student_id,s.word,s.phoneme,s.class_idx,s.correct,s.mfcc,s.confidence) ) conn.commit(); conn.close() return {'stored': len(samples)} @app.get('/api/model') def get_model(): with open('/app/model/weights_latest.json') as f: return json.load(f) @app.post('/api/sessions') def log_session(logs: List[SessionLog]): conn = psycpg2.connect(DB) cur = conn.cursor() for l in logs: cur.execute( 'INSERT INTO sessions ' '(student_id,unit,stage,phoneme,attempts,correct,accuracy) ' 'VALUES (%s,%s,%s,%s,%s,%s,%s)', (l.student_id,l.unit,l.stage,l.phoneme,l.attempts,l.correct,l.accuracy) ) conn.commit(); conn.close() return {'logged': len(logs)}
```

Systemd service — keeps FastAPI running

Create `/etc/systemd/system/early.service`:

```
[Unit] Description=EARLY FastAPI After=network.target postgresql.service [Service]
User=early WorkingDirectory=/app/api
Environment=DATABASE_URL=postgresql://earlyuser:password@localhost/earlydb
ExecStart=/app/api/venv/bin/uvicorn main:app --host 127.0.0.1 --port 8000 Restart=always
[Install] WantedBy=multi-user.target

sudo systemctl daemon-reload sudo systemctl enable early sudo systemctl start early sudo
systemctl status early # confirm running
```

Step 8 — Nginx Configuration

Create `/etc/nginx/sites-available/early`:

```
server { listen 80; server_name early.gregtutors.com; # frontend static files root
/app/early/dist; index index.html; location / { try_files $uri $uri/ /index.html; } #
backend – proxy to FastAPI location /api/ { proxy_pass http://127.0.0.1:8000;
proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr; } # model weights –
static JSON location /model/ { alias /app/model/; } }

sudo ln -s /etc/nginx/sites-available/early /etc/nginx/sites-enabled/ sudo rm
/etc/nginx/sites-enabled/default # remove default site sudo nginx -t # test config sudo
systemctl reload nginx
```

Step 9 — SSL Certificate (Let's Encrypt)

■ DNS must be fully propagated before this step or certbot will fail. Confirm with: `nslookup early.gregtutors.com` — must return your IONOS VPS IP.

```
sudo certbot --nginx -d early.gregtutors.com # certbot will: # - issue a free SSL
certificate from Let's Encrypt # - automatically update your Nginx config for HTTPS # -
set up auto-renewal # confirm auto-renewal sudo certbot renew --dry-run
```

■ After certbot runs, `early.gregtutors.com` is live on HTTPS. The iPad can now access the app and the microphone API will work.

Step 10 — Nightly Retraining Cron

Create `/app/api/retrain.py`:

```
import psycopg2, numpy as np, json, os from sklearn.neural_network import MLPClassifier DB
= os.environ['DATABASE_URL'] conn = psycopg2.connect(DB) cur = conn.cursor() cur.execute(
'SELECT mfcc, class_idx FROM mfcc_samples WHERE correct = true' ) rows = cur.fetchall() if
len(rows) < 20: print(f'Only {len(rows)} samples – skipping retrain') exit() X =
np.array([r[0] for r in rows]) y = np.array([r[1] for r in rows]) model = MLPClassifier(
hidden_layer_sizes=(16, 8), max_iter=500, random_state=42 ) model.fit(X, y) weights = {
'w': [l.tolist() for l in model.coefs_], 'b': [l.tolist() for l in model.intercepts_] }
with open('/app/model/weights_latest.json', 'w') as f: json.dump(weights, f)
```

```
cur.execute('UPDATE model_versions SET active = false') cur.execute( 'INSERT INTO
model_versions (sample_count, weights_path, active)' ' VALUES (%s, %s, true)', (len(rows),
'/app/model/weights_latest.json') ) conn.commit() print(f'Retrained on {len(rows)}
samples')
```

Add to crontab:

```
crontab -e # add this line: 0 2 * * *
DATABASE_URL=postgresql://earlyuser:password@localhost/earlydb /app/api/venv/bin/python
/app/api/retrain.py >> /app/logs/retrain.log 2>&1
```

Step 11 — Deploy Script

Push from Cursor, SSH in and run this. One command updates the live site.

```
# /app/deploy.sh #!/bin/bash set -e echo 'Pulling latest...' cd /app/early && git pull
origin main echo 'Restarting API...' sudo systemctl restart early echo 'Done.
early.gregtutors.com updated.'

chmod +x /app/deploy.sh # run from your Windows machine ssh early@early.gregtutors.com
'/app/deploy.sh'
```

Step 12 — Update Client API URL

The only change needed in the frontend code. No CORS configuration required — frontend and backend share the same domain.

```
// one constant at the top of your JS const API = 'https://early.gregtutors.com'; // flush
session samples (once per session, not per word) await fetch(API + '/api/samples', {
method: 'POST', headers: { 'Content-Type': 'application/json' }, body:
JSON.stringify(sessionBuffer) }); // load model weights at session start const weights =
await fetch(API + '/api/model').then(r => r.json()); net.loadWeights(weights);
```

Setup Checklist

#	Step	Done?
1	Add A record in GoDaddy DNS for early.gregtutors.com	
2	Reinstall VPS — Ubuntu 22.04 LTS (IONOS)	
3	First login — create early user, secure SSH	
4	apt install all dependencies + firewall	
5	PostgreSQL — create db, user, and tables	
6	Create /app directory structure, clone repo	
7	FastAPI main.py + systemd service	
8	Nginx config — frontend + API proxy	

9	Confirm DNS propagated, run certbot SSL	
10	retrain.py + cron job (2am nightly)	
11	deploy.sh script	
12	Update client API constant, cancel Vercel	

Cost Summary

Item	Cost
IONOS VPS	existing — ~\$6/month
gregtutors.com (GoDaddy)	existing
SSL certificate	free — Let's Encrypt
PostgreSQL	free — self-hosted on VPS
Vercel	cancelled
Total	~\$6/month fixed

EARLY full VPS setup guide. DNS: GoDaddy (ns75/ns76.domaincontrol.com). Host: IONOS VPS. OS: Ubuntu 22.04 LTS. Domain: early.gregtutors.com. No Vercel dependency after migration.